

Storyboard Técnico — Gerenciador de Currículos ("TalentBase")

****Documento:**** Storyboard técnico e arquitetural

****Front-end analisado:**** `Job\_Propecting\_Dashboard.zip` (export React/TypeScript gerado pelo Figma Make)

****Back-end proposto:**** C# / ASP.NET Core Web API

****Banco de dados:**** relacional (SQL Server ou PostgreSQL)

0. Contexto e premissas (o que foi encontrado no anexo)

O ZIP não contém imagens do Figma, e sim o ****código React/TypeScript exportado**** do protótipo (Figma Make), nomeado ****"Job Prospecting Dashboard"***** (marca interna no app: ****TalentBase****). Stack identificada: React 18 + TypeScript + Vite + Tailwind CSS 4 + Radix UI/shadcn + `recharts` + `lucide-react`.

Isso permitiu extrair fielmente as telas, campos e regras ****já desenhadas****, em vez de supor genericamente. Três pontos moldam todo o restante deste documento:

1. ****O protótipo cobre só o lado RH.**** Existem 4 telas (Dashboard, Candidatos, Vagas, Cadastrar Currículo), todas atrás de um usuário fixo "Equipe de RH" — não há tela de login, nem qualquer fluxo de candidato. Como o requisito pede autenticação por ****dois perfis****, todo o módulo de candidato (portal, login, área "meus dados") é ****proposto neste documento****, não extraído do Figma.
2. ****"Vaga pretendida" é texto livre, sem vínculo real com uma vaga.**** No formulário de cadastro, o campo é um `<input>` com autocomplete (`datalist`) sobre os títulos de vaga — não existe uma chave estrangeira `vagald`. Ou seja, hoje ****Candidato e Vaga não têm relação relacional real****, só uma coincidência de texto. Isso é corrigido na modelagem de dados (seção 4) com a entidade `Candidatura`.
3. ****"Tempo de experiência" é uma faixa fixa em texto ("4 anos", "10+ anos"), não uma data.**** O campo do formulário é um `<select>` com 11 opções pré-definidas. Isso afeta diretamente a regra de ranqueamento (seção 5), que precisa reconciliar esse dado de baixa precisão com um cálculo confiável.
4. ****Não há entidade "Empresa" na UI.**** O sistema parece mono-tenant (uso interno de uma única empresa — veja `rh@empresa.com.br` fixo no rodapé da sidebar). Como o requisito pede a entidade explicitamente, ela é modelada como tabela de primeira classe — o que também deixa o sistema pronto para evoluir para multi-empresa (ex.: uma agência de recrutamento atendendo vários clientes) sem retrabalho.

Essas são as premissas assumidas para o restante do documento. Se alguma não refletir a intenção real do projeto, é só sinalizar — o impacto de cada uma está isolado nas seções correspondentes.

1. Fluxos principais de usuário

1.1 Fluxo do RH (já mapeado no protótipo)

1. **Login** (perfil RH) → redireciona para o **Dashboard**.
2. **Dashboard**: visão geral (KPIs, gráfico "Candidatos por Mês", funil de status, últimas 5 candidaturas) → ponto de partida para navegar a Candidatos ou Vagas.
3. **Gerenciar Vagas**: criar/editar/encerrar uma vaga (tela nova — o protótipo só lista) → vaga aberta passa a aparecer no portal público do candidato.
4. **Gerenciar Candidatos**: buscar/filtrar por nome, status ou departamento → alterar status inline (dropdown) → adicionar observações → excluir/arquivar.
5. **Cadastro manual de currículo**: quando o RH recebe um currículo por fora do sistema (e-mail, indicação, LinkedIn) → preenche o formulário em 3 seções (pessoal, profissional, observações) → candidato entra com status **Novo**.
6. **Ranquear candidatos de uma vaga**: RH abre uma vaga específica e visualiza os candidatos ordenados por tempo de experiência (e outros critérios), para priorizar quem entrevistar primeiro (seção 5).

Máquina de estados da candidatura (já implícita no protótipo, formalizada aqui):

...

Novo → Em Análise → Entrevista → Aprovado

→ Reprovado

...

`Aprovado` e `Reprovado` são estados finais. `Reprovado` pode ocorrer a partir de qualquer estágio anterior (não precisa passar por Entrevista).

1.2 Fluxo do candidato (proposto — não existe no protótipo)

1. Acessa a **vitrine pública de vagas** (sem login) → filtra por cargo, departamento, localização, modalidade → abre o detalhe de uma vaga.
2. **Cria conta / login** (perfil Candidato) — pode ser motivado pelo botão "Candidatar-se".
3. Completa o **perfil**: dados pessoais, **experiências profissionais** (empresa, cargo, data início/fim — a fonte real do tempo de experiência), habilidades, upload do currículo em PDF.
4. **Candidata-se** a uma vaga específica (cria um registro de `Candidatura`, diferente do cadastro manual do RH, que hoje não gera esse vínculo).
5. Acompanha o **status da candidatura** em "Minhas Candidaturas" (somente leitura) — mesma máquina de estados da seção 1.1.
6. Recebe notificação (e-mail, e opcionalmente in-app) a cada mudança de status ou convite de entrevista.

2. Principais telas e componentes (Figma)

2.1 Já existentes no protótipo (visão RH)

| Tela | Componentes principais |

|---|---|

| **Dashboard** | 4 KPI cards (Total de Candidatos, Vagas Abertas, Entrevistas Agendadas, Aprovados no Mês); gráfico de barras "Candidatos por Mês" (candidatos vs. contratados, `recharts`); "Funil de Status" (barra de progresso por status + taxa de aprovação); lista "Candidaturas Recentes" (últimas 5) |

| **Candidatos** | Busca por texto (nome/vaga/e-mail); filtro por status; filtro por departamento; tabela com avatar, vaga/depto., localização, experiência, status (dropdown editável inline), data, ação de excluir; contador de resultados e resumo por status no rodapé |

| **Vagas** | Resumo (Abertas / Em Processo / Encerradas); grid de cards de vaga (título, departamento, tipo CLT/PJ, status, localização, data de publicação, nº de vagas, nº de candidatos, avatar stack) |

| **Cadastrar Currículo** | Formulário em 3 seções — (1) Dados Pessoais: nome*, e-mail*, telefone, cidade/UF; (2) Dados Profissionais: vaga pretendida* (autocomplete), departamento* (select), tempo de experiência (select de faixas), habilidades (tags separadas por vírgula); (3) Observações (textarea) — com validação inline e tela de sucesso |

Navegação: sidebar fixa desktop (Dashboard / Candidatos / Vagas / Cadastrar Currículo) + versão compacta mobile (ícones no header).

2.2 A projetar (extensão necessária para atender candidato + RH completos)

| Tela | Por quê é necessária |

|---|---|

| **Login / Cadastro** (com roteamento por perfil) | Não existe nenhuma tela de autenticação hoje |

| **Portal do candidato — Vitrine de vagas** (pública) | Ponto de entrada do candidato; filtros por cargo/depto./local/modalidade |

| **Detalhe da vaga + "Candidatar-se"** | Hoje `JobsView` só lista cards; falta a tela de detalhe e a ação de candidatura |

| **Meu Perfil / Currículo** (candidato) | Dados pessoais + CRUD de experiências profissionais (essencial para o ranqueamento real) + upload de PDF |

| **Minhas Candidaturas** (candidato) | Acompanhamento de status, hoje só visível ao RH |

| **Detalhe do Candidato** (RH — drill-down) | Hoje a única ação é o dropdown de status na lista; falta ver histórico, experiências, anexos, notas do recrutador |

| ****Criar/Editar Vaga**** (RH) | `JobsView` atual só exibe vagas — não há formulário de criação/edição |

| ****Ranking de candidatos por vaga**** (RH) | Tela nova, resposta direta ao requisito de ranqueamento (seção 5) |

2.3 Reaproveitamento de design system

O front-end já traz Radix UI/shadcn (`accordion`, `dialog`, `select`, `tabs`, `table`, `dropdown-menu`, `avatar`, `sonner` para toasts, etc.) totalmente integrados ao Tailwind. Todas as telas novas da seção 2.2 devem reaproveitar esses mesmos componentes e os tokens de tema (`src/styles/theme.css`) para manter consistência visual, em vez de introduzir uma segunda biblioteca de UI.

3. Requisitos de autenticação

****Estratégia geral:**** ASP.NET Core Identity (gestão de usuários/senhas) + ****JWT**** (access + refresh token) para autenticar chamadas da SPA React à Web API — separação clássica front-end desacoplado / API stateless.

****Perfis e permissões****

- Roles do Identity: `RH`, `Candidato` (e opcionalmente `Admin`, para gerenciar contas de RH e dados da Empresa).
- Cadastro de ****Candidato****: self-service, público (`POST /api/auth/registrar`).
- Cadastro de ****RH****: ****não**** é self-service — conta criada/convidada por um Admin, já que RH tem acesso a dados pessoais de todos os candidatos (superfície sensível).
- Autorização por atributo nos controllers: `[Authorize(Roles = "RH")]`, `[Authorize(Roles = "Candidato")]`; endpoints de leitura de vagas abertas ficam públicos (`[AllowAnonymous]`).

****Fluxo de tokens****

- Login retorna ****access token**** (JWT, TTL curto — 15 a 60 min) + ****refresh token**** (TTL longo — 7 a 30 dias), este último armazenado com hash no banco (tabela `RefreshToken`) para permitir revogação.
- Access token guardado em memória no front-end (não em `localStorage`, para reduzir superfície de XSS); refresh token em cookie `httpOnly` + `Secure` + `SameSite=Strict`.
- `POST /api/auth/refresh` renova o access token sem exigir novo login; logout revoga o refresh token no banco.

****Complementares****

- HTTPS obrigatório em todos os ambientes; CORS restrito às origens conhecidas do front-end.
- Política de senha (mínimo de caracteres, complexidade) via `IdentityOptions`.
- Rate limiting no endpoint de login (proteção contra força bruta) — `Microsoft.AspNetCore.RateLimiting` (nativo desde .NET 7+).

- ****LGPD****: tela de cadastro do candidato precisa de consentimento explícito de tratamento de dados; prever endpoint de exclusão de conta/dados a pedido do titular (``DELETE /api/candidatos/me``) e política de retenção documentada.

- 2FA para contas RH é uma extensão recomendável (não obrigatória no MVP), dado o volume de dados sensíveis que essas contas acessam.

4. Modelagem de dados

4.1 Entidades principais

| Tabela | Campos-chave | Relacionamentos |

| --- | --- | --- |

| `Empresa` | Id, Nome, CNPJ, Setor, Site | 1:N com `Vaga` |

| `Usuario` | Id, Nome, Email, SenhaHash, EmpresaId (nullable) | 1:1 opcional com `Candidato`; N:1 com `Empresa` (quando RH) |

| `Departamento` | Id, Nome | 1:N com `Vaga`; 0:1 com `Candidato` (área de interesse) |

| `Candidato` | Id, UsuarioId (nullable), Nome, Email, Telefone, Cidade, Estado, CurriculumUrl, PosicaoDesejada, ExperienciaTotalMeses (calculado) | 1:N com `Experiencia`; N:N com `Habilidade`; 1:N com `Candidatura` |

| `Experiencia` | Id, CandidatoId, NomeEmpresa, Cargo, DataInicio, DataFim (nullable) | N:1 com `Candidato` |

| `Vaga` | Id, EmpresaId, DepartamentoId, Titulo, Localizacao, Modalidade, Tipo, VagasDisponiveis, ExperienciaMinimaMeses, Status, DataPublicacao | N:1 com `Empresa`/`Departamento`; 1:N com `Candidatura`; N:N com `Habilidade` |

| `Habilidade` | Id, Nome (unique) | N:N com `Candidato` e `Vaga` |

| `Candidatura` | Id, CandidatoId, VagaId, Status, DataCandidatura, Observacoes | N:1 com `Candidato` e `Vaga`; 1:N com `HistoricoStatusCandidatura`; único por (CandidatoId, VagaId) |

| `HistoricoStatusCandidatura` | Id, CandidaturaId, StatusAnterior, StatusNovo, AlteradoPorUsuarioId, AlteradoEm | N:1 com `Candidatura` |

****Tabelas de junção / extensões opcionais:**** `CandidatoHabilidade` e `VagaHabilidade` (N:N puras); `Entrevista` (Id, CandidaturaId, DataHora, Local/Link, EntrevistadorUsuarioId, Status, Feedback) — útil, mas não bloqueia o MVP.

4.2 Decisões de modelagem que valem destacar

- ****`Empresa` (quem contrata) ≠ `Experiencia.NomeEmpresa` (empregador anterior do candidato).*** São conceitos diferentes: a primeira é uma entidade do sistema (tabela `Empresa`, quem publica vagas); a segunda é texto livre dentro de `Experiencia`, já que normalizar os empregadores anteriores dos candidatos como entidades próprias seria engenharia excessiva para o escopo atual.

- **`Candidatura`** corrige a falta de FK real entre Candidato e Vaga observada no protótipo (campo ``position`` era texto livre). Com isso, perguntas como "quantos candidatos se aplicaram à vaga X" passam a ser uma consulta confiável, não uma comparação de strings.
- **Banco de talentos:** ``Candidato`` pode existir sem nenhuma ``Candidatura`` formal — os campos ``PosicaoDesejada`` (texto) e ``DepartamentoInteresseld`` no próprio candidato preservam o fluxo de cadastro manual e rápido que já existe no protótipo. Quando o RH vincula formalmente o candidato a uma vaga, cria-se o registro em ``Candidatura``.
- **Enums no banco usam identificadores sem acentuação** (``EmAnalise``, ``Hibrido``) — o rótulo acentuado ("Em Análise") fica só na camada de apresentação (front-end).

4.3 Exemplo de classes C# / EF Core

```

`csharp
// Models/Candidato.cs

public class Candidato
{
    public int Id { get; set; }
    public int? Usuariold { get; set; }
    public Usuario? Usuario { get; set; }

    public string Nome { get; set; } = string.Empty;
    public string Email { get; set; } = string.Empty;
    public string? Telefone { get; set; }
    public string? Cidade { get; set; }
    public string? Estado { get; set; }
    public string? CurriculumUrl { get; set; }

    public string? PosicaoDesejada { get; set; }
    public int? DepartamentoInteresseld { get; set; }

    // Cache calculado a partir de Experiencias — ver seção 5
    public int ExperienciaTotalMeses { get; set; }

    public ICollection<Experiencia> Experiencias { get; set; } = new List<Experiencia>();
    public ICollection<Habilidade> Habilidades { get; set; } = new List<Habilidade>();
    public ICollection<Candidatura> Candidaturas { get; set; } = new List<Candidatura>();

```

```
public DateTime CriadoEm { get; set; } = DateTime.UtcNow;  
}
```

```
// Models/Experiencia.cs
```

```
public class Experiencia  
{  
    public int Id { get; set; }  
    public int Candidatoid { get; set; }  
    public Candidato Candidato { get; set; } = null!;  
  
    public string NomeEmpresa { get; set; } = string.Empty; // texto livre (empregador anterior)  
    public string Cargo { get; set; } = string.Empty;  
    public DateOnly DataInicio { get; set; }  
    public DateOnly? DataFim { get; set; } // null = emprego atual  
    public string? Descricao { get; set; }  
}
```

```
// Models/Vaga.cs
```

```
public class Vaga  
{  
    public int Id { get; set; }  
    public int Empresaid { get; set; }  
    public Empresa Empresa { get; set; } = null!;  
    public int Departamentoid { get; set; }  
    public Departamento Departamento { get; set; } = null!;  
  
    public string Titulo { get; set; } = string.Empty;  
    public string Localizacao { get; set; } = string.Empty;  
    public ModalidadeTrabalho Modalidade { get; set; }  
    public TipoContrato Tipo { get; set; }  
    public int VagasDisponiveis { get; set; }  
    public int ExperienciaMinimaMeses { get; set; }  
    public string Descricao { get; set; } = string.Empty;  
    public StatusVaga Status { get; set; } = StatusVaga.Aberta;
```

```

public DateOnly DataPublicacao { get; set; }

public ICollection<Candidatura> Candidaturas { get; set; } = new List<Candidatura>();
public ICollection<Habilidade> HabilidadesRequeridas { get; set; } = new List<Habilidade>();
}

// Models/Candidatura.cs
public class Candidatura
{
    public int Id { get; set; }
    public int CandidatoId { get; set; }
    public Candidato Candidato { get; set; } = null!;
    public int VagaId { get; set; }
    public Vaga Vaga { get; set; } = null!;

    public StatusCandidatura Status { get; set; } = StatusCandidatura.Novo;
    public DateTime DataCandidatura { get; set; } = DateTime.UtcNow;
    public string? Observacoes { get; set; }

    public ICollection<HistoricoStatusCandidatura> Historico { get; set; } = new List<HistoricoStatusCandidatura>();
}
...

```csharp
// Data/AppDbContext.cs (trecho relevante)
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
 modelBuilder.Entity<Candidatura>()
 .HasIndex(c => new { c.CandidatoId, c.VagaId })
 .IsUnique(); // um candidato não se candidata duas vezes à mesma vaga

 modelBuilder.Entity<Candidato>()
 .HasMany(c => c.Habilidades)
 .WithMany(h => h.Candidatos)

```



```

 .UsingEntity(j => j.ToTable("CandidatoHabilidade"));

modelBuilder.Entity<Vaga>()
 .HasMany(v => v.HabilidadesRequeridas)
 .WithMany(h => h.Vagas)
 .UsingEntity(j => j.ToTable("VagaHabilidade"));
}
...

```

## ## 5. Regras de ranqueamento por tempo de experiência

### ### 5.1 Como medir "tempo de experiência"

Duas fontes possíveis, e o time precisa decidir entre elas (ou oferecer as duas):

- **Caminho A — recomendado:** substituir o `<select>` de faixas fixas do formulário atual por um CRUD real de `Experiencia` (empresa, cargo, data início/fim). O tempo total é **calculado**, não digitado — muito mais preciso, e é o que sustenta um ranqueamento confiável.
- **Caminho B — atalho de curto prazo:** manter o `<select>` de faixas como está hoje e mapear cada faixa para um valor numérico mínimo em meses ("Menos de 1 ano" → 0, "5 anos" → 60, "10+ anos" → 120). Zero mudança de UI, porém menos preciso (não captura múltiplas experiências nem sobreposição de períodos).

Recomendação prática: **lançar com B e migrar para A** assim que o portal do candidato existir — o modelo de dados da seção 4 já suporta os dois sem retrabalho (o campo `ExperienciaTotalMeses` em `Candidato` é populado por qualquer um dos dois caminhos).

### ### 5.2 Cálculo (caminho A) — cuidado com sobreposição de períodos

Somar `DataFim - DataInicio` de cada experiência ingenuamente **infla o resultado** quando o candidato teve dois vínculos simultâneos (ex.: CLT + freelance no mesmo período). A forma correta é unir os intervalos antes de somar:

```

```csharp
public static class ExperienciaCalculator
{
    public static int CalcularMesesTotais(IEnumerable<Experiencia> experiencias)
```

```

{
    var intervalos = experiencias
        .Select(e => (Inicio: e.DataInicio, Fim: e.DataFim ?? DateOnly.FromDateTime(DateTime.Today)))
        .OrderBy(i => i.Inicio)
        .ToList();

    var unificados = new List<(DateOnly Inicio, DateOnly Fim)>();
    foreach (var atual in intervalos)
    {
        if (unificados.Count > 0 && atual.Inicio <= unificados[^1].Fim)
        {
            var ultimo = unificados[^1];
            unificados[^1] = (ultimo.Inicio, atual.Fim > ultimo.Fim ? atual.Fim : ultimo.Fim);
        }
        else
        {
            unificados.Add(atual);
        }
    }

    return unificados.Sum(i => (i.Fim.Year - i.Inicio.Year) * 12 + (i.Fim.Month - i.Inicio.Month));
}
}
...

```

`Candidato.ExperienciaTotalMeses` deve ser ****recalculado e persistido**** a cada `POST`/`PUT`/`DELETE` em `/api/candidatos/{id}/experiencias` (não recalculado a cada leitura — mais barato para ordenar listas grandes).

5.3 Regra de ranqueamento (MVP)

Para a listagem de candidatos de uma vaga específica (`GET /api/vagas/{id}/ranking`):

1. Candidatos com `ExperienciaTotalMeses >= Vaga.ExperienciaMinimaMeses` aparecem primeiro, ordenados de forma decrescente pela experiência.

2. Candidatos abaixo do mínimo ****não são descartados automaticamente**** — aparecem depois, sinalizados como "abaixo do requisito"; a decisão final é do RH.

```
```csharp
[HttpGet("{id}/ranking")]
[Authorize(Roles = "RH")]
public async Task<ActionResult<List<CandidatoRankingDto>>> GetRanking(int id)
{
 var vaga = await _context.Vagas.FindAsync(id);
 if (vaga is null) return NotFound();

 var candidaturas = await _context.Candidaturas
 .Where(c => c.VagaId == id)
 .Include(c => c.Candidato)
 .ToListAsync();

 var ranking = candidaturas
 .Select(c => new CandidatoRankingDto
 {
 CandidatoId = c.CandidatoId,
 Nome = c.Candidato.Nome,
 ExperienciaMeses = c.Candidato.ExperienciaTotalMeses,
 AtendeRequisito = c.Candidato.ExperienciaTotalMeses >= vaga.ExperienciaMinimaMeses,
 Status = c.Status
 })
 .OrderByDescending(c => c.AtendeRequisito)
 .ThenByDescending(c => c.ExperienciaMeses)
 .ToList();

 return Ok(ranking);
}
```
```

5.4 Evolução futura — score composto

Quando fizer sentido considerar mais do que experiência (habilidades batendo com a vaga, recência da candidatura), uma pontuação ponderada substitui o `OrderByDescending` simples:

...

```
Score = (peso_exp × min(exp_candidato / exp_minima_vaga, 2.0))
      + (peso_skill × habilidades_em_comum / habilidades_pedidas)
      + (peso_recencia × fator_tempo_desde_candidatura)
```

...

Os pesos podem ficar configuráveis por vaga ou globais. Não é necessário para o MVP — vale registrar como próximo passo natural.

6. Endpoints de API (C# / ASP.NET Core)

`AuthController` — `/api/auth`

| Método | Rota | Descrição | Acesso |
|--------|---------------------------|---|----------------------|
| POST | /api/auth/registrar | Cadastro público (perfil Candidato) | Público |
| POST | /api/auth/login | Autentica; retorna access + refresh token | Público |
| POST | /api/auth/refresh | Renova access token | Refresh token válido |
| POST | /api/auth/logout | Revoga refresh token | Autenticado |
| POST | /api/auth/esqueci-senha | Dispara e-mail de redefinição | Público |
| POST | /api/auth/redefinir-senha | Confirma nova senha via token | Público |

`UsuariosController` — `/api/usuarios` (contas RH)

| Método | Rota | Descrição | Acesso |
|--------|--------------------|----------------------------|--------|
| GET | /api/usuarios | Lista contas RH da empresa | Admin |
| POST | /api/usuarios | Convida/cria conta RH | Admin |
| PUT | /api/usuarios/{id} | Atualiza dados | Admin |

| DELETE | `/api/usuarios/{id}` | Desativa conta | Admin |

`EmpresasController` — `/api/empresas`

| Método | Rota | Descrição | Acesso |

| --- | --- | --- | --- |

| GET | `/api/empresas/{id}` | Detalhe da empresa | RH |

| PUT | `/api/empresas/{id}` | Atualiza dados da empresa | Admin |

`CandidatosController` — `/api/candidatos`

| Método | Rota | Descrição | Acesso |

| --- | --- | --- | --- |

| GET | `/api/candidatos` | Lista/busca/filtra (`nome`, `status`, `departamento`, `vagaId`, `experienciaMin`, `page`, `pageSize`) | RH |

| GET | `/api/candidatos/{id}` | Detalhe | RH |

| POST | `/api/candidatos` | Cadastro manual (equivale à tela "Cadastrar Currículo" atual) | RH |

| PUT | `/api/candidatos/{id}` | Atualiza dados | RH |

| DELETE | `/api/candidatos/{id}` | Remove/arquiva | RH |

| GET | `/api/candidatos/me` | Meu perfil | Candidato |

| PUT | `/api/candidatos/me` | Atualiza meu perfil | Candidato |

| POST | `/api/candidatos/me/curriculo` | Upload do PDF do currículo | Candidato |

`ExperienciasController` — `/api/candidatos/{candidatoId}/experiencias`

| Método | Rota | Descrição | Acesso |

| --- | --- | --- | --- |

| GET | `/` | Lista experiências do candidato | RH ou dono |

| POST | `/` | Adiciona experiência (recalcula `ExperienciaTotalMeses`) | RH ou dono |

| PUT | `/ {id}` | Edita experiência | RH ou dono |

| DELETE | `/ {id}` | Remove experiência | RH ou dono |

`VagasController` — `/api/vagas`

| Método | Rota | Descrição | Acesso |

|---|---|---|---|

| GET | `/api/vagas` | Lista vagas abertas (filtros: `departamento`, `localizacao`, `modalidade`, `tipo`) | Público |

| GET | `/api/vagas/{id}` | Detalhe da vaga | Público |

| POST | `/api/vagas` | Cria vaga | RH |

| PUT | `/api/vagas/{id}` | Editar vaga | RH |

| PATCH | `/api/vagas/{id}/status` | Abre/encerra vaga | RH |

| DELETE | `/api/vagas/{id}` | Remove vaga | RH |

| GET | `/api/vagas/{id}/candidatos` | Lista candidaturas da vaga | RH |

| GET | `/api/vagas/{id}/ranking` | ★ Candidatos ordenados pela regra da seção 5 | RH |

`CandidaturasController` — `/api/candidaturas`

| Método | Rota | Descrição | Acesso |

|---|---|---|---|

| GET | `/api/candidaturas` | Lista com filtros (`vagal`, `candidatoid`, `status`) | RH |

| GET | `/api/candidaturas/me` | Minhas candidaturas | Candidato |

| POST | `/api/candidaturas` | Candidata-se a uma vaga (`vagal` no corpo) | Candidato |

| GET | `/api/candidaturas/{id}` | Detalhe | RH ou dono |

| PATCH | `/api/candidaturas/{id}/status` | Muda status (grava em `HistoricoStatusCandidatura`) | RH |

| GET | `/api/candidaturas/{id}/historico` | Histórico de status | RH ou dono |

`DashboardController` — `/api/dashboard`

| Método | Rota | Descrição | Acesso |

|---|---|---|---|

| GET | `/api/dashboard/resumo` | KPIs (total de candidatos, vagas abertas, entrevistas, aprovados) | RH |

| GET | `/api/dashboard/candidaturas-por-mes` | Série mensal para o gráfico de barras | RH |

| GET | `/api/dashboard/funil-status` | Contagem de candidaturas por status | RH |

`HabilidadesController` — `/api/habilidades`

| Método | Rota | Descrição | Acesso |

|---|---|---|---|

| GET | `/api/habilidades?busca=` | Autocomplete de habilidades cadastradas | Autenticado |

| POST | `/api/habilidades` | Cria nova habilidade (se ainda não existir) | RH ou Candidato |

7. Bibliotecas e ferramentas recomendadas

7.1 Back-end (C# / .NET)

| Biblioteca | Uso |

|---|---|

| **ASP.NET Core Web API (.NET 10, LTS atual)** | Framework base da API |

| **Entity Framework Core** | ORM; provider `Microsoft.EntityFrameworkCore.SqlServer` ou `Npgsql.EntityFrameworkCore.PostgreSQL` |

| **ASP.NET Core Identity** | Gestão de usuários, senhas, roles |

| **Microsoft.AspNetCore.Authentication.JwtBearer** | Emissão/validação de JWT |

| **FluentValidation** | Validação de DTOs mais expressiva que DataAnnotations |

| **AutoMapper** (ou Mapster) | Mapeamento Entity ↔ DTO |

| **Serilog** | Logging estruturado |

| **Swashbuckle.AspNetCore** | Swagger/OpenAPI — documentação automática dos endpoints da seção 6 |

| **xUnit + FluentAssertions + Moq** | Testes unitários e de integração |

| **Hangfire** (opcional) | Jobs em background — reenvio de e-mails, recomputo em lote de ranking |

7.2 Front-end (o que já está no `package.json` do export + recomendações)

| Biblioteca | Situação |

|---|---|

| React 18 + TypeScript + Vite | Já em uso |

| Tailwind CSS 4 + Radix UI/shadcn | Já em uso — reaproveitar nas telas novas (seção 2.3) |

| `recharts`, `lucide-react`, `sonner` | Já em uso (gráficos, ícones, toasts) |

| `react-hook-form` | **Já está no `package.json`, mas não é usado** — `RegisterView` valida com `useState` manual. Recomenda-se migrar para `react-hook-form` + `zod` ao construir os formulários novos (login, perfil, criação de vaga) |

| `react-router` | **Também já está no `package.json`, mas não é usado** — a navegação hoje troca de tela via `useState<View>`. Migrar para rotas reais é necessário para dar URL própria ao portal do candidato, suportar deep-link e botão "voltar" |

| Axios ou `fetch` + **TanStack Query (React Query)** | Não está no projeto ainda; necessário para consumir a API C#, com cache e invalidação automática |

7.3 Infraestrutura

- Banco relacional: **SQL Server** ou **PostgreSQL** (ambos com bom suporte a EF Core).
- Armazenamento de currículos em PDF: Azure Blob Storage ou AWS S3 (não em disco local/banco).
- Containerização do back-end com Docker; pipeline CI/CD (GitHub Actions, já que o repositório usa GitHub pelo `codeload.github.com`/`raw.githubusercontent.com` como padrão comum, ou Azure DevOps).

8. Ordem recomendada de implementação

****Fase 0 — Fundação****

1. Setup do projeto .NET Web API + EF Core + banco relacional (migração inicial).
2. Criar as tabelas núcleo: `Empresa`, `Usuario`, `Departamento`, `Candidato`, `Experiencia`, `Vaga`, `Habilidade`, `Candidatura`, `HistoricoStatusCandidatura`.
3. Configurar ASP.NET Identity + JWT + roles (`RH`, `Candidato`).
4. Swagger ativo; CORS liberado para a origem do front-end.

****Fase 1 — Autenticação e núcleo RH****

5. Endpoints de `AuthController` (registro de candidato, login, refresh).
6. CRUD de `Vaga` (RH).
7. CRUD de `Candidato` — cadastro manual, espelhando o `RegisterView` já existente.
8. Ligar o front-end atual (telas Cadastrar Currículo e Vagas) à API real, substituindo os dados mockados (`initialCandidates`, `jobsData`).

****Fase 2 — Pipeline de candidaturas****

9. `CandidaturasController` completo (aplicar-se, listar, mudar status) + `HistoricoStatusCandidatura`.
10. Ligar `CandidatesView` (mudança de status inline) à API.
11. Endpoints do `DashboardController` + ligar `DashboardView` aos dados reais.

****Fase 3 — Portal do candidato****

12. Telas novas: login/cadastro do candidato, vitrine pública de vagas, detalhe da vaga com "Candidatar-se".

13. Tela "Meu Perfil" + `ExperienciasController` (CRUD de experiências) — é o que habilita o cálculo real do tempo de experiência (caminho A da seção 5.1).

14. Upload de currículo (integração com o storage de blobs).

****Fase 4 — Ranqueamento e refinamentos****

15. Implementar `ExperienciaCalculator` e persistir `Candidato.ExperienciaTotalMeses` a cada alteração de experiência.

16. Endpoint `GET /api/vagas/{id}/ranking` (seção 5.3) + tela de ranking no lado RH.

17. Normalizar habilidades (`Habilidade`, `CandidatoHabilidade`, `VagaHabilidade`) com autocomplete no front.

****Fase 5 — Produção****

18. Testes automatizados dos fluxos críticos (login, candidatura, mudança de status, ranking).

19. Observabilidade (Serilog + ferramenta de monitoramento, ex. Application Insights).

20. Deploy (Docker + CI/CD).

21. Checklist LGPD: consentimento no cadastro, endpoint de exclusão de dados, política de retenção documentada.

9. Observações finais

Duas decisões, sinalizadas ao longo do documento, valem uma validação explícita com o time antes de codar:

- ****Mono-tenant vs. multi-empresa**** (seção 0, item 4) — a entidade `Empresa` já suporta ambos; só muda se o cadastro de empresa fica travado em um único registro ou vira uma tela de gestão.

- ****Faixa de experiência vs. datas reais**** (seção 5.1) — recomendado começar pelo caminho B (mapear a faixa atual para meses) e migrar para o caminho A assim que o CRUD de experiências existir; o campo `ExperienciaTotalMeses` no `Candidato` já foi desenhado para não exigir migração de schema quando isso acontecer.

Este documento cobre o suficiente para iniciar a Fase 0 imediatamente; o nível de detalhe de telas/endpoints das Fases 3–4 pode ser refinado conforme o portal do candidato for tomando forma.